

Escopo - Projeto “EsToDoList do Estudante”

Objetivo:

- O objetivo desse projeto é ajudar os estudantes a se organizarem com tarefas, provas, trabalhos, prazos e outras atividades diárias. Essa ferramenta melhoraria a rotina dos estudantes, gerando menos estresse e ajudando o desempenho.

Requisitos Funcionais (RF):

- O usuário adicionaria uma nova tarefa ao clicar no símbolo de “+” . Ele precisaria adicionar um nome para a tarefa, e poderia adicionar uma descrição, data e hora para realizar a tarefa, incentivando a autogestão para cumprir tarefas dentro do prazo.
- Para editar uma tarefa já criada, o usuário precisaria passar o mouse em cima da tarefa e clicar com o botão esquerdo do mouse para habilitar as funções de edição de texto. Ele poderia alterar o nome, a descrição, a data e a hora.
- Para excluir uma tarefa, teria um símbolo de uma lixeira ao clicar na tarefa. O sistema pedirá uma confirmação para ter certeza de que o usuário realmente deseja excluir a tarefa e, em caso afirmativo, excluiria a tarefa e seus dados (como nome, data, hora, etc).
- Ao concluir uma tarefa, o usuário poderia clicar em uma caixinha de seleção que teria antes do texto de cada tarefa, assinalando ela. O sistema entenderia essa tarefa como realizada e o texto teria um traço em cima e ficaria com uma cor mais escura, simbolizando que aquela atividade já foi realizada.
- Caso o usuário deseje encontrar uma tarefa específica, ele poderia digitar o nome da tarefa na barra de pesquisa ou digitar a data que deveria realizá-la para poder encontrar uma tarefa em específico.

Requisitos Não Funcionais (RNF):

- A usabilidade do software é importante, pois ele deve ser de fácil compreensão para não gerar estresse aos usuários.
- A confiabilidade é importante para que as tarefas e datas não fiquem confusas ou se misturem.
- Desempenho é importante, pois o software deve responder às interações com menos de 2 segundos, permitindo que o utilizador crie ou edite tarefas sem demora.

Fora de Escopo:

- Nessa primeira versão do projeto, não teremos Inteligência Artificial para ajudar a criar textos e descrições para as tarefas, pois os próprios estudantes devem descrever o que precisarão fazer e adicionar ou remover tarefas, visando melhorar a própria organização e habilidade de priorizar atividades.
- Também não teremos a possibilidade de compartilhar as listas de tarefas, pois é importante priorizar a individualidade da pessoa e estimular a capacidade de descrever tarefas diárias de forma autoral.

Metodologia Cascata:

Requisitos:

- Escrever o documento de escopo com todas as funcionalidades;
- Entrevistar alunos para entender como eles organizam suas tarefas hoje.
- Mapear os requisitos não funcionais (como tempo de resposta e segurança dos dados);
- Definir os critérios de aceitação para o cadastro e edição de tarefas.

Análise e projeto:

- Desenhar as telas do aplicativo no Figma;
- Definir a paleta de cores e a fonte que serão usadas no site.
- Criar o diagrama de fluxo do usuário (*user flow*) navegando pelas telas;
- Modelar a estrutura do banco de dados (ou estrutura de dados) para armazenar as tarefas.

Desenvolvimento:

- Escrever o código HTML da página principal;

- Programar a função em JavaScript que salva uma nova tarefa no navegador.
- Criar os estilos CSS para tornar a interface responsiva em dispositivos móveis;
- Implementar a funcionalidade de exclusão e marcação de tarefa como concluída.

Testes:

- Verificar se o aplicativo funciona corretamente nos navegadores Chrome e Firefox;
- Tentar “quebrar” o campo de data, inserindo um texto em vez de um número.
- Testar a usabilidade com um grupo pequeno de alunos para avaliar a facilidade de uso;
- Validar se as tarefas salvas continuam disponíveis após recarregar a página.

Implantação e manutenção:

- Publicar a versão final do site em um servidor online para que todos possam usar;
- Corrigir um bug reportado por um usuário uma semana após o lançamento.
- Configurar o domínio personalizado para acesso ao sistema;
- Monitorar o desempenho do servidor e aplicar atualizações periódicas de segurança.

Matriz de risco

PROBABILIDADE	ALTA = 3	média	alta	alta						
	MÉDIA = 2	baixa	média	alta						
	BAIXA = 1	baixa	baixa	média						
		BAIXO = 1	MÉDIO = 2	ALTO = 3			RISCO	PROBABILIDADE	IMPACTO	PESO
		IMPACTO					EXEMPLO 1	1	3	4
							EXEMPLO 2	1	1	2
							EXEMPLO 3	2	3	5

* FEITO NO EXCEL *

LEGENDA:

-  aceitar
-  observar
-  monitorar
-  mitigar

Risco (descrição)	Probabilidade (baixa/alta)	Impacto (baixo/alto)	Plano de ação (o que faremos para prevenir ou remediar?)
Ex: o único programador do projeto fica doente e se ausenta por uma semana (risco de recursos)	Baixa	Alto	Manter toda a documentação do projeto atualizada e salva em um local compartilhado (como o Notion), para que outra pessoa possa entender o documento
Risco 1: E se todo código desenvolvido em uma aula fosse perdido porque ninguém fez commit ou enviou o projeto no GitHub?	Baixa	Alto	Criar o hábito de fazer commits ou ativar o salvamento automático do código
Risco 2: Internet indisponível no momento da entrega/apresentação	Alta	Alto	Baixar os conteúdos no pendrive ao final do dia ou entregar a atividade antes do prazo
Risco 3: Durante o desenvolvimento do EsToDoList, os alunos que testaram o sistema gostaram da ideia e começaram a pedir um chat para conversar sobre as tarefas	Alta	Alto	Mudar o código para inserir o chat como uma nova atualização entre outras melhorias, além de conversar com a equipe para ver se as mudanças são possíveis dentro do prazo